

Alejandro Rodríguez

Máster Universitario en Sistemas Inteligentes (MUSI)
Especialidad en Inteligencia Artificial y Ciencia de Datos
Universitat de les Illes Balears

Curso 2025/26



Universitat
de les Illes Balears

Tutor(es): Gabriel Moyà Alcover y Miquel Miró Nicolau

- 1 Problema y objetivo
- 2 Datos y preprocesado
- 3 Denoising Autoencoder
- 4 Arquitectura
- 5 Entrenamiento
- 6 Inferencia y score
- 7 Calibración y evaluación
- 8 Adaptación, reproducibilidad y conclusiones

Planteamiento

En lugar de enseñar al modelo todas las patologías posibles, se aprende únicamente cómo son los cortes **normales**.

$$s(x_N) < s(x_A), \quad x_N \sim p_N, \quad x_A \not\sim p_N.$$

- Los pesos se entrenan solo con `train/good`.
- Una anomalía es una imagen incompatible con la normalidad aprendida.
- El sistema no identifica una enfermedad concreta ni sustituye un diagnóstico.

¿Puede un **denoising autoencoder**, entrenado solo con cortes FLAIR **normales**, separar cortes normales de anómalos en BraTS2021_slice mediante el error de reconstrucción?

- Aprendizaje de pesos: no utiliza imágenes anómalas.
- Selección de mejor época: validación normal.
- Calibración del umbral: validación normal + anómala.
- Evaluación final: test sin reajustar el modelo ni el umbral.

Partición	Normales	Anómalas	Uso
Entrenamiento	7.500	—	ajuste de pesos
Validación	39	44	mejor época y umbral
Test	640	3.075	evaluación final

- Cortes 2D FLAIR en PNG monocromo.
- Redimensionado de 240×240 a 128×128 .
- Conversión a float32 y normalización a $[0, 1]$.
- Tensor final por imagen: $1 \times 128 \times 128$.
- Sin data augmentation geométrica.

El fondo de la resonancia es prácticamente negro y no aporta información útil para detectar anomalías cerebrales.

$$M = \mathbb{I}(x > 0,01)$$

- El umbral 0,01 separa fondo casi negro de primer plano.
- El ruido de entrenamiento se aplica solo donde $M = 1$.
- La MSE también se calcula solo sobre el primer plano.
- Así el modelo no obtiene una pérdida artificialmente baja por reconstruir fondo negro.

Autoencoder convencional

$$x \longrightarrow f_{\theta}(x) \approx x$$

- Puede aproximarse demasiado a la identidad.
- Una red potente podría reconstruir también anomalías.

Denoising Autoencoder

$$\tilde{x} \longrightarrow f_{\theta}(\tilde{x}) \approx x$$

- La entrada se corrompe artificialmente.
- El objetivo es recuperar la anatomía normal limpia.
- La tarea obliga a utilizar contexto espacial.

Basado en la idea de Kaschenas et al. [1].

$$n_s \sim \mathcal{N}(0, I)_{16 \times 16}, \quad \tilde{x} = x + 0,2 M \odot \text{roll}(\text{up}(n_s))$$

- ❶ Generar ruido gaussiano a baja resolución: 16×16 .
- ❷ Interpolarlo bilinealmente a 128×128 .
- ❸ Desplazarlo circularmente en posiciones aleatorias.
- ❹ Aplicarlo solo al primer plano cerebral.

Por qué ruido grueso

Produce perturbaciones espaciales suaves y de mayor escala; eliminar ruido píxel a píxel sería una tarea más local y potencialmente más trivial.

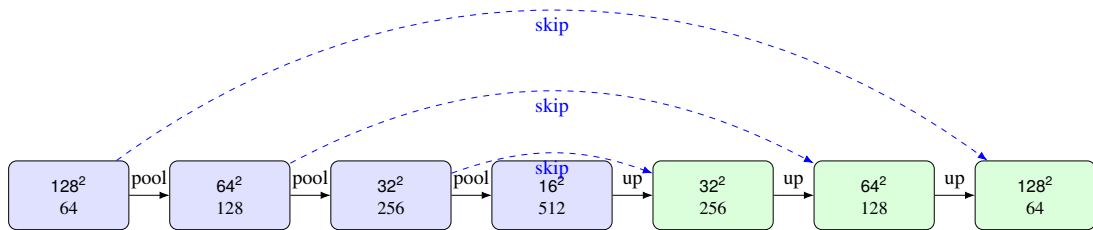
La red reconstruye la imagen limpia:

$$\hat{x} = f_{\theta}(\tilde{x})$$

y optimiza una MSE de primer plano:

$$\mathcal{L}(\theta) = \frac{\sum M \odot (\hat{x} - x)^2}{\max(\sum M, 1)}.$$

- Es una regresión píxel a píxel.
- El cuadrado penaliza especialmente errores grandes.
- El fondo se excluye para centrar el aprendizaje en tejido cerebral.



- Cuatro niveles: 64, 128, 256 y 512 canales.
- Tres reducciones por AvgPool1d(2).
- Tres subidas bilineales + convolución.
- Skip connections entre escalas equivalentes.
- Salida final $1 \times 128 \times 128$ mediante convolución 1×1 .
- **8.558.721 parámetros entrenables.**

Cada ConvBlock aplica dos veces:

$$\text{Conv}_{3 \times 3} \rightarrow \text{SiLU} \rightarrow \text{GroupNorm}(8).$$

Elemento	Motivo
Conv. 3×3	Extraer patrones espaciales locales con pocos parámetros
padding=1	Mantener altura y anchura dentro de cada bloque
SiLU	Introducir una no linealidad suave
GroupNorm	Normalizar sin depender de estadísticas entre ejemplos del batch
Average pooling	Reducir resolución de forma suave
Skip connections	Recuperar detalle espacial para la reconstrucción
Salida lineal	Regresión de intensidad sin saturación de una sigmoide

Ejemplo en la primera etapa del decoder:

$$d_3 = \text{dec3}(\text{cat}(\text{up3}(e_4), e_3)) .$$

- e_4 : representación profunda a $16 \times 16 \times 512$.
- up3 : la convierte en $32 \times 32 \times 256$.
- e_3 : conserva detalle del encoder a $32 \times 32 \times 256$.
- La concatenación se hace en canales: $256 + 256 = 512$.
- dec3 transforma esos 512 canales en 256.

Idea

El decoder combina contexto profundo con información espacial de alta resolución.

Hiperparámetros

- Imagen: 128×128 .
- Batch: 16.
- Épocas: 50.
- Semilla: 42.
- Adam + AMSGrad.
- Learning rate: 10^{-4} .
- Weight decay: 10^{-5} .
- Ruido: $\sigma = 0,2$, resolución 16.

Control experimental

- Python, NumPy y PyTorch con semilla fija.
- cuDNN determinista.
- Mejor época por MSE en valid/good.
- Umbral fijado después con validación etiquetada.
- Test evaluado sin reajustes.

Para cada lote:

`zero_grad` $\rightarrow$ `forward` $\rightarrow \mathcal{L}$ $\rightarrow$ `backward` $\rightarrow$ `optimizer.step`.

- **Adam**: adapta el tamaño de actualización usando momentos del gradiente.
- **AMSGrad**: variante de Adam que conserva un máximo histórico del segundo momento.
- **Weight decay**: regulariza los pesos.
- **CosineAnnealingLR**: $T_{\text{máx}} = 100$.

Detalle del código actual

`scheduler.step()` se ejecuta por lote, por lo que $T_{\text{máx}} = 100$ está expresado en **actualizaciones**, no en épocas.

Después de cada época se evalúan únicamente imágenes normales de validación.

$$e^* = \arg \min_e \text{MSE}_{\text{valid/good}}(e)$$

- Se entrenan las 50 épocas.
- Cuando mejora la MSE normal de validación se guarda una copia de los pesos.
- Al terminar, se recargan los pesos de la mejor época.
- Las imágenes anómalas no participan en esta selección.

En inferencia se introduce la imagen **limpia**; no se añade ruido.

$$e(x) = |x - f_{\theta}(x)|$$

Después:

- 1 Refinar la máscara con average pooling 5×5 y umbral 0,95.
- 2 Eliminar el error fuera del interior cerebral.
- 3 Aplicar mediana local 5×5 .

$$A(x) = f_{\text{med},5} \left(\tilde{M} \odot |x - f_{\theta}(x)| \right).$$

El filtro de mediana reduce respuestas puntuales aisladas.

BMAD proporciona una etiqueta por corte, por lo que el mapa debe reducirse a un único número:

$$s(x) = \max_{i,j} A(x)_{ij}.$$

- Una anomalía puede ocupar una región pequeña.
- Un promedio global podría diluirla entre miles de píxeles normales.
- El máximo pregunta si existe alguna región con error especialmente alto.
- La mediana previa evita que un único píxel ruidoso domine el score.

Con los scores y etiquetas de **validación** se calcula la curva ROC. El código selecciona:

$$t^* = \arg \max_t (TPR(t) - FPR(t)) .$$

Es el criterio de Youden:

$$J = \text{sensibilidad} + \text{especificidad} - 1.$$

Regla metodológica

El threshold se selecciona una sola vez en validación y después se aplica sin cambios al test.

AUROC

Evalúa la capacidad del score para ordenar anómalas por encima de normales sin depender de un threshold concreto. 0,5 equivale al azar y 1 a separación perfecta.

Balanced accuracy

$$BA = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

Es adecuada porque el test contiene 640 normales y 3.075 anómalas.

Principios conservados

- DAE tipo U-Net.
- Tres reducciones y skips.
- Ruido gaussiano grueso.
- 16×16 y $\sigma = 0,2$.
- MSE de primer plano.
- Error absoluto y filtro mediana.

Adaptaciones del proyecto

- Un canal FLAIR.
- Cortes 2D independientes.
- Score escalar por imagen.
- Interpolación bilineal + convolución.
- GroupNorm con 8 grupos.
- AUROC y balanced accuracy.
- Threshold de Youden en validación.

La implementación separa responsabilidades:

- `data.py`: carga, particiones y preprocesado.
- `dae.py`: arquitectura U-Net del DAE.
- `experiment.py`: entrenamiento, score, threshold y métricas.
- `train.py`: interfaz de ejecución.
- Notebook Colab: GPU, descarga de datos, smoke test y entrenamiento completo.

Cada ejecución guarda pesos, configuración, historial, scores individuales, threshold, métricas y reconstrucciones.

Partición	AUROC	BA
Validación	0,9452	0,8820
Test	0,9633	0,9097

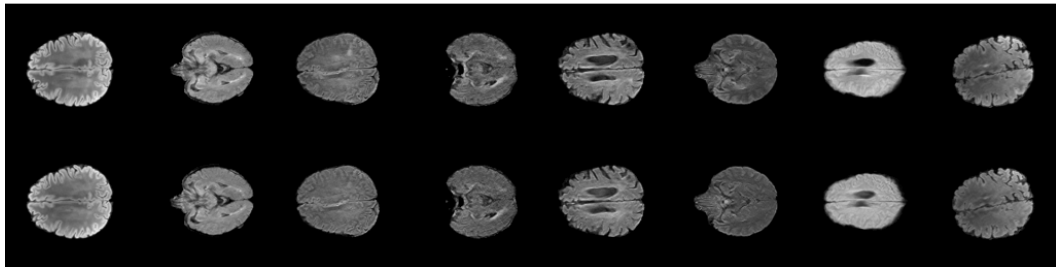
- 50 épocas; seed 42.
- Época seleccionada: 47.
- Threshold: 0,18611.
- Tiempo: 1,58 h en GPU.

Test: 3.715 cortes

Sensibilidad: 0,9210; especificidad: 0,8984.

TP: 2.832; TN: 575; FP: 65; FN: 243.

DAE original (arriba) / reconstrucción (abajo)



Entrada, reconstrucción y discrepancias locales usadas por el score.

- Una modalidad FLAIR 2D pierde información multimodal y tridimensional.
- Una etiqueta por corte no permite evaluar localización de lesión.
- El máximo espacial puede ser sensible a artefactos locales.
- La validación es pequeña: 39 normales y 44 anómalas.
- El threshold usa etiquetas de validación: los pesos son no supervisados, pero la decisión binaria final está calibrada supervisadamente.
- Un benchmark no demuestra generalización clínica externa.

- Se implementó un DAE tipo U-Net entrenado **solo con normales**.
- El ruido grueso convierte el aprendizaje en una tarea de restauración, no en una simple copia de la entrada.
- El error de reconstrucción se filtra y reduce a un score por corte.
- Pesos, calibración y test están metodológicamente separados.
- En test: AUROC 0,9633 y balanced accuracy 0,9097.

Trabajo futuro: varias semillas, ablaciones de hiperparámetros, información multimodal/3D, máscaras de lesión y validación externa.

¡Gracias!

Alejandro Rodríguez

Detección no supervisada de anomalías mediante un DAE



Antanas Kascenas, Nicolas Pugeault, and Alison Q. O'Neil.

Denoising autoencoders for unsupervised anomaly detection in brain MRI.

In *Proceedings of the 5th Conference on Medical Imaging with Deep Learning*, volume 172 of *Proceedings of Machine Learning Research*, pages 653–664, 2022.